CorpusLab Technical Handbook

Product Overview

CorpusLab is a corpus workbench for teams building retrieval-augmented generation systems. The product helps engineers inspect sources, extracted documents, chunks, embeddings, retrieval runs, trace timelines, evidence summaries, evals, and reports across arbitrary knowledge sources: product docs, support knowledge bases, policies, contracts, research papers, technical specifications, code documentation, wikis, and resumes.

The current implementation is privacy-first and local by default. Uploaded binaries are not persisted. Extracted text, chunk text, metadata, embeddings, retrieval runs, traces, evals, and reports are stored in Postgres. Retrieval and trace diagnosis use local lexical scoring and a local hash embedding baseline so relevance can be debugged without external model calls.

Repository Structure

- `apps/api`: Axum API service, runtime config, routes, startup state, telemetry, and integration tests.
- `apps/web`: React, Vite, TypeScript workbench UI with Home, Corpus, Retrieval, Trace Debugger, Eval Lab, CI Runs, Audit Reports, and Settings pages.
- `crates/core`: shared contracts for projects, sources, documents, chunks, ingestion, embeddings, retrieval, traces, evals, reports, and config.
- `crates/rag`: extraction, chunking, document intelligence, embeddings, retrieval scoring, trace construction, and eval metrics.
- `crates/storage`: repository traits plus in-memory and SQLx/Postgres storage implementations.
- `migrations`: SQLx migrations for projects, sources, documents, chunks, embeddings, retrieval runs, traces, and evals.
- `docs`: architecture, development, testing, privacy, ingestion, retrieval, traces, Eval Lab, roadmap, ADRs, and this handbook.

Rust Workspace Architecture

The Rust workspace keeps domain contracts separate from implementation. `crates/core` owns serializable types and IDs. `crates/rag` owns deterministic RAG behavior. `crates/storage` owns bounded persistence traits and adapters. `apps/api` composes those crates into HTTP routes.

`crates/storage/src/repository.rs` separates health, project, source, document, embedding, retrieval, trace, eval, auth, and CI eval capabilities. `AppRepository` is the application-facing composite. The narrow `IngestionRepository` compatibility boundary contains no methods of its own and composes only the project, source, and document capabilities required by synchronous uploads.

This shape lets future hosted services, local collectors, workers, and GPU/HPC indexing processes reuse the same contracts without coupling them to Axum route code.

React App Architecture

The web app lives in `apps/web/src`.

- `App.tsx` maps the public site, auth pages, and workbench routes.
- `layouts/MarketingLayout.tsx`, `layouts/AuthLayout.tsx`, and `layouts/WorkbenchLayout.tsx` separate public, authentication, and application chrome.
- `features/marketing` owns the public product narrative. The landing route is lazy-loaded and decomposed under `features/marketing/landing` into hero, failure story, retrieval demo, capability story, product tour, enterprise trust, and CTA sections.
- `features/auth` owns the login and signup entry surfaces.
- `components/brand` owns the CorpusLab mark and wordmark.
- `components/ui` owns reusable marketing and product primitives such as buttons, feature cards, pricing cards, and product mockups. Landing-specific interaction state remains local to its section instead of expanding shared primitives prematurely.
- `components/workbench` owns authenticated-workbench primitives: page headers, recoverable empty states, workflow guides, panels, toolbars, status pills, and metric cards. These primitives keep dense corpus, retrieval, trace, Eval Lab, report, and settings surfaces visually consistent without creating a standalone design-system package.
- `lib/apiClient.ts` is a compatibility barrel for the API boundary. New code should prefer domain exports under `lib/api`, such as `lib/api/sources`, `lib/api/retrieval`, `lib/api/traces`, and `lib/api/evalLab`.
- Domain files under `pages` are thin route wrappers or compatibility re-exports. Product implementation belongs under `features`, following `docs/frontend-architecture.md`.
- `pages/OverviewPage.tsx` and `pages/SettingsPage.tsx` are thin compatibility exports; their implementations live under `features/workbench/home` and `features/workbench/settings`.
- `features/workbench/sources` owns Corpus upload, the document library, and focused document/chunk inspection at `/app/sources/:documentId`.
- `features/workbench/retrieval` owns the question-first retrieval test. A domain hook coordinates source, embedding, query, and trace mutations; focused panels own query, filter,

and embedding controls; result components own evidence summaries, citations, and ranking details.

- `features/workbench/traces` owns the searchable Trace Debugger and focused `/app/traces/:traceId` debugger. A domain hook owns trace loading and tab state; separate components own summary, failure labels, evidence metrics, timeline spans, reruns, and Eval Lab case creation.
- `features/workbench/eval-lab` owns Eval Lab datasets, experiments, gates, and the focused `/app/evals?view=ci-runs` quality-automation view.
- `features/workbench/reports` owns audit report creation, generated report lists, focused `/app/reports/:reportId` detail, privacy classification, and Markdown copy. Existing CI failures, run diagnoses, and corpus findings remain visible as report candidates.
- `features/workbench/settings` owns Workspace, API keys, Runtime, and Privacy tabs.

The authenticated workbench follows the guided workflow documented in `docs/guided-workbench.md`. Home derives a live setup checklist from `/api/v1/overview`. A typed shell manifest defines the canonical Setup, Debug, Quality, Share, and Admin navigation groups, active parent routes, linked breadcrumbs, help copy, and product-loop ordering. Shared page-header, workflow-guide, panel, toolbar, status, metric, and empty-state primitives keep hierarchy, density, and first-time-user orientation consistent, while route errors remain inside a recoverable workbench boundary.

Generated `apps/web/dist` files should not be edited by hand. Run `cd apps/web && npm run build`.

Public Marketing Runtime

The `/` route is a separate Vite chunk loaded through `React.lazy`. This keeps Motion and landing-only CSS out of authenticated workbench startup. `LazyMotion` loads `domAnimation`, while `MotionConfig` honors the user's reduced-motion preference. Motion constants live in `features/marketing/landing/motion.ts`; interactive display fixtures and their TypeScript contracts live in `landingData.ts`.

Landing sections use independent CSS modules and stable media aspect ratios. Animations change only opacity and transforms. The hero bitmap is requested eagerly, while product-tour and diagnosis images load lazily. The production gate enforces combined gzip limits of 180 KB for JavaScript and 20 KB for CSS through `npm run size:check`.

See `docs/marketing-experience.md` for interaction ownership, accessibility behavior, screenshot generation, and visual regression checks.

API Route Reference

Route composition lives in `apps/api/src/http/routing.rs`; `apps/api/src/http/mod.rs` only declares handler modules and exports the router. Protected workbench routes share session middleware without changing their `/api/v1` paths.

All handler errors serialize as `{ "error": { "code", "message" } }`. Expected client errors retain specific messages, while internal storage failures are sanitized to prevent infrastructure details from crossing the API boundary. The web API client parses this envelope and keeps raw response text only for diagnostics.

- `GET /healthz`: process liveness.
- `GET /readyz`: readiness; checks database connectivity when storage is configured.
- `GET /api/v1/config`: safe product/runtime config for the web app.
- `POST /api/v1/auth/signup`: create a local user, organization, workspace, membership, and session.
- `POST /api/v1/auth/login`: verify local credentials and issue an HttpOnly session cookie.
- `POST /api/v1/auth/logout`: revoke the current session.
- `GET /api/v1/auth/me`: return the authenticated user, organization, workspace, and role.
- `GET /api/v1/workspaces/current`: return the active workspace context.
- `GET /api/v1/api-keys`: list workspace API keys without secrets.
- `POST /api/v1/api-keys`: create a workspace API key and return the one-time `clab_...` secret.
- `DELETE /api/v1/api-keys/:api_key_id`: revoke an API key.
- `POST /api/v1/sources/files`: multipart ingestion for text, Markdown, HTML, and PDF.
- `GET /api/v1/sources`: sources with document and chunk counts.
- `GET /api/v1/documents/:document_id/chunks`: persisted chunks ordered by ordinal.
- `GET /api/v1/embeddings/status`: embedding model and indexing coverage.
- `POST /api/v1/embeddings/index`: synchronously indexes stored chunks with the local provider.
- `POST /api/v1/retrieval/query`: lexical, vector, or hybrid retrieval with evidence summary and citations.
- `GET /api/v1/traces`: recent trace summaries.
- `GET /api/v1/traces/:trace_id`: full trace timeline.
- `POST /api/v1/traces/from-retrieval-run`: save a retrieval run as a trace.
- `POST /api/v1/traces/:trace_id/rerun`: rerun a trace with changed retrieval settings.
- `GET /api/v1/retrieval/evals`: saved eval cases.
- `POST /api/v1/retrieval/evals`: create an eval case.
- `POST /api/v1/retrieval/evals/run`: run eval cases for a retrieval mode.
- `GET /api/v1/eval-lab/datasets`: list Eval Lab datasets.
- `POST /api/v1/eval-lab/datasets`: create an Eval Lab dataset.

- `GET /api/v1/eval-lab/datasets/:dataset_id`: load a dataset with cases.
- `POST /api/v1/eval-lab/evidence/query`: resolve selected evidence IDs and search bounded document/chunk candidates.
- `POST /api/v1/eval-lab/datasets/:dataset_id/cases`: add a case to a dataset.
- `PATCH /api/v1/eval-lab/cases/:case_id`: update a case.
- `DELETE /api/v1/eval-lab/cases/:case_id`: delete a case.
- `POST /api/v1/eval-lab/experiments`: run a dataset across selected retrieval modes.
- `GET /api/v1/eval-lab/experiments`: list recent experiments.
- `GET /api/v1/eval-lab/experiments/:experiment_id`: load one experiment.
- `POST /api/v1/eval-lab/experiments/:experiment_id/compare`: compare selected modes.
- `POST /api/v1/eval-lab/ci/runs`: run an Eval Lab dataset from CI using an API key.
- `GET /api/v1/eval-lab/ci/runs`: list CI eval runs.
- `GET /api/v1/eval-lab/ci/runs/:run_id`: load one CI eval run.
- `GET /api/v1/eval-lab/ci/runs/:run_id/report`: load the export-ready CI gate report.

Database Schema And Migrations

Migrations are in `migrations`.

Core tables include `organizations`, `workspaces`, `users`, `workspace_memberships`, `auth_sessions`, `api_keys`, `projects`, `sources`, `ingestion_runs`, `documents`, `chunks`, `chunk_embeddings`, `retrieval_playground_runs`, `retrieval_playground_hits`, `debug_traces`, `trace_rerun_experiments`, `retrieval_eval_datasets`, `retrieval_eval_cases`, `retrieval_eval_runs`, `retrieval_eval_results`, `retrieval_eval_experiments`, and `ci_eval_runs`.

The Eval Lab evidence-search migration enables `pg_trgm` and adds GIN indexes for normalized source names, document paths, chunk section titles, and chunk text. UUID primary keys continue serving direct requested-ID resolution. The migration changes indexes only and does not rewrite corpus records.

Recent metadata additions:

- `documents.document_profile`: one of `general`, `technical_docs`, `policy_or_legal`, `support_kb`, `research_paper`, `code_docs`, or `resume`.
- `documents.extraction_quality`: `high`, `medium`, `low`, or `unknown`.
- `documents.warnings`: extraction and quality warnings.
- `chunks.quality_flags`: heading-only, too-short, duplicate, low-density, extraction-warning, and evidence-candidate flags.
- `chunks.text_density`, `chunks.is_duplicate`, and `chunks.evidence_score_hint`.

Run migrations with `just db-migrate`.

File Ingestion Pipeline

The browser sends multipart files to `POST /api/v1/sources/files`. The API validates file count, per-file bytes, total request bytes, and extension allowlist using typed runtime config. The first workflow is synchronous so behavior is simple to observe.

For each file:

- Detect a supported format.
- Extract readable text with the Rust extractor.
- Compute file checksum.
- Detect document profile and extraction quality.
- Chunk text with structured document chunking or whitespace fallback.
- Annotate chunk quality and duplicate signals.
- Persist source, ingestion run, document, and chunk metadata.
- Return document results and preview chunks.

Original binaries are not stored.

Extraction Pipeline

`crates/rag/src/extraction.rs` owns extraction.

- Text and Markdown use UTF-8 extraction.
- HTML uses best-effort readable text extraction.
- PDF uses embedded text extraction through `pdf-extract`.
- OCR is intentionally not included yet.

Failures are structured as unsupported type, invalid UTF-8, PDF extraction failure, empty text, size limit, or storage failure.

Chunking Strategies

`structured` is the default. It generalizes the earlier resume-focused strategy into a document-aware structured chunker.

Structured chunking:

- Detects headings in technical docs, policies, support KBs, research papers, code docs, resumes, and general documents.
- Preserves paragraph and bullet group boundaries where possible.
- Packs blocks up to `target_tokens`.
- Uses whitespace fallback when one block exceeds the token limit.
- Applies overlap only for token-limit splits within the same section.

`whitespace` remains available for debugging and deterministic fallback comparisons.

Document Intelligence

`crates/rag/src/intelligence.rs` adds deterministic document analysis.

Profiles:

- `general`
- `technical_docs`
- `policy_or_legal`
- `support_kb`
- `research_paper`
- `code_docs`
- `resume`

Chunk quality flags identify weak evidence candidates before retrieval ranking promotes them. Heading-only chunks and duplicates are visible in the UI and suppressed from strong evidence summaries.

Embedding Provider System

`crates/rag/src/embedding.rs` defines the local embedding provider boundary. The current provider is `local-hash-v1` with configurable dimension. It is deterministic, local, and CPU-friendly. It is not intended to be the final semantic model.

The provider boundary is intentionally ready for future implementations:

- local ONNX embedding model
- CUDA worker
- Metal worker
- remote enterprise embedding service
- batch indexing queue

Retrieval Scoring

`crates/rag/src/retrieval.rs` implements the local baseline retriever. Candidate retrieval, answerability assessment, and extractive answer construction are separate modules so broad debugging results cannot silently become citations.

Modes:

- `lexical`: term overlap, phrase matches, section boosts, path boosts, and metadata boosts.
- `vector`: cosine similarity over stored embeddings.
- `hybrid`: semantic similarity plus lexical and metadata signals.

The response includes raw and normalized score breakdowns:

- semantic
- lexical
- phrase
- section
- path
- metadata

Quality upgrades:

- Deduplicates hits by checksum and normalized text.
- Tracks `duplicate_count`.
- Adds `quality_flags`.
- Adds `evidence_strength`.
- Avoids promoting heading-only and weak chunks as strong answer evidence.
- Assesses direct body-text support per hit; semantic, path, section, source, and metadata signals remain retrieval-only.
- Produces an evidence summary only from supported hits and returns `insufficient_evidence` with no citations when candidates do not pass the gate.

The default answerability policy requires at least `0.50` distinct query-term coverage and two matching terms in one body sentence. One-term queries require that term, and all numeric query terms must occur in the same supporting sentence. These defaults are typed, validated, and configurable. Persisted assessments contain statuses, reasons, and counts rather than copied query or body text.

Trace Debugger

`crates/rag/src/tracing.rs` converts a saved retrieval response into an inspectable trace.

Trace records include:

- query input
- retrieval mode and latency
- ordered spans
- ranked evidence and citations
- deterministic failure labels
- rerun comparisons

Current spans are query input, retrieval ranking, evidence summary, eval check, and optional generation metadata. The Eval Check span is present even before eval linkage so the product can teach users where regression checks fit in the workflow.

`crates/rag/src/diagnosis.rs` is the single deterministic analysis boundary. It classifies each run as strong, mixed, weak, or failing; records typed failures and affected evidence; explains every score from persisted components; and maps failures to prioritized remediation areas. It detects weak and duplicate evidence, heading-only chunks, answerability gaps, semantic-only and metadata-only candidates, missing or partial embeddings, low top-two score margin, semantic/lexical disagreement, citation grounding problems, and Eval Lab expected-evidence gaps. The low-margin ratio defaults to `0.10` and is exposed through validated debugger configuration.

New retrieval responses, traces, eval case results, rerun comparisons, and audit reports snapshot this diagnosis. Existing JSON remains readable through optional serde defaults. Legacy trace labels remain populated from the structured diagnosis for `/api/v1` compatibility.

The stable guarantees for retrieval responses, trace diagnosis, rerun comparisons, Eval Lab metrics, gates, and local-first behavior are defined in `docs/rag-invariants.md`. Synthetic regression corpora and expected outcomes live under `fixtures/`; typed Rust tests remain the contract-level source of truth while those fixtures provide reviewable scenarios for API, UI, and future SDK tests.

Postgres stores trace summaries in `debug_traces`, full trace timelines in `trace_json`, and rerun comparisons in `trace_rerun_experiments`. The memory store supports the same API for tests and local no-Docker sessions.

The web UI exposes `/app/traces` as a searchable saved-run list. `/app/traces/:traceId` displays diagnosis, ranked evidence, ordered spans, and rerun comparison without overloading the list view. The Retrieval page exposes `Debug this run`, which saves the current retrieval response and navigates directly to that focused detail route.

Eval System

Eval Lab is the primary quality-control workflow. Datasets group cases by product area, customer workflow, release gate, or corpus. Cases store a query plus expected chunk IDs, document IDs, or both. Experiments run a dataset across selected retrieval modes and freeze a config snapshot with `top_k`, scoring weights, embedding model metadata, and dataset case count.

Metrics include recall@k, precision@k, MRR, top hit rank, citation coverage, weak evidence count, missing embedding failures, and latency p50/p95. Failure labels include expected evidence missing, correct document wrong chunk, low precision, weak evidence, missing embeddings, heading-only evidence, and duplicate evidence.

The default gate passes when average recall@k is at least `0.80`, critical missing-embedding failures are absent, and weak-evidence cases are at or below 20%. Failed gates appear in Mission Control and point users to `/app/evals`.

Eval Lab v2 adds regression read models over saved experiments. Dataset history and trend endpoints compare the latest run with the previous compatible baseline for the same dataset, `top_k`, and retrieval-mode set. Experiment detail surfaces gate movement, metric deltas, newly failed cases, recovered cases, changed top evidence, and changed failure labels. Audit reports created from experiments include that regression context when available while preserving metadata-only privacy.

Expected-evidence lookup has a dedicated bounded storage contract. Explicit document and chunk IDs resolve directly in deduplicated request order and do not consume candidate limits. Additional documents and chunks use independent limits, deterministic ordering, and Postgres trigram indexes over source names, document paths, section titles, and chunk text. Chunk results contain a 280-character Unicode-safe preview and a truncation flag rather than unrestricted body text. The workbench submits searches only through Search or Enter while selection changes continue resolving selected metadata through cancellable React Query requests.

Legacy cases may retain expected-evidence IDs whose source data was deleted or is no longer accessible. Case updates preserve omitted evidence arrays without revalidation, while present arrays replace and validate the corresponding selection. The workbench tracks evidence changes as normalized ID sets, exposes explicit stale-item removal, sends empty arrays only for intentional clearing, and restores persisted drafts on cancellation. Validation completes before the merged case is stored, so failed repairs cannot partially persist unrelated edits.

The same PATCH contract distinguishes case-note omission, replacement, and clearing: an omitted `notes` property preserves the stored value, a string replaces it, and `null` clears it. Successful edits synchronously replace the matching case in the dataset query cache before background invalidation, so an immediate reopen cannot restore pre-save scalar or evidence values from a stale prop.

The legacy `/api/v1/retrieval/evals` endpoints remain compatible for older flows. Trace Debugger saves cases into Eval Lab only after the user chooses a dataset and explicitly marks expected document/chunk evidence.

Report Contracts

`crates/core/src/report.rs` defines the additive RAG Audit Report contract. `DebugReport` freezes workspace/project ownership, source identity, a privacy-filtered subject, executive summary, deterministic context metadata, findings, recommendations, evidence references, and an RFC3339 creation timestamp.

Reports can originate from a trace, Eval Lab experiment, CI eval run, or manual investigation. Findings link stable failure-label codes to labeled evidence references. Recommendations use typed remediation areas for chunking, embeddings, `top_k`, retrieval mode, reranking, metadata filters, citations, and corpus coverage.

Report privacy is distinct from project privacy. `metadata_only` excludes query and document content, `snippets_allowed` permits explicitly approved bounded text, and `full_local_only` cannot be shared or exported without an explicit privacy downgrade. The full workflow and sharing rules are documented in `docs/rag-audit-reports.md`.

The legacy `RetrievalReport`, `RetrievalDiagnosis`, and `EvidenceIssue` contracts remain available for compatibility. Audit reports are additive and do not replace existing retrieval, trace, Eval Lab, CI, or diagnostic-queue contracts.

`crates/rag/src/reports` builds deterministic reports from traces, Eval Lab experiments, and CI eval runs. Build context supplies IDs, ownership, privacy mode, and timestamp. Source builders freeze configuration and comparison metadata, map failure labels to stable findings and remediation categories, deduplicate recommendations, and apply report privacy before evidence enters the report.

`ReportRepository` provides workspace-scoped save, list, and detail operations with MemoryStore/Postgres parity. Postgres stores one append-only `debug_reports` row per snapshot, including indexed workspace/project/source/privacy columns and canonical report JSON. Duplicate report IDs fail; multiple snapshots from the same source are allowed.

Protected `/api/v1/reports` routes create reports from traces, experiments, and native CI runs, then expose workspace-scoped list/detail and Markdown export. Handlers authenticate the active session to determine report ownership. Metadata-only is the request default, CI ownership is verified, and full-local reports return `422` from export.

The Reports feature owns the shared creation action used by Trace Detail, Eval experiment detail, and failed CI gate rows. Every source integration opens a privacy confirmation panel, defaults to

metadata-only, guards against duplicate submission, and navigates to the persisted report detail after creation.

`crates/rag/src/reports/markdown.rs` renders paid-audit-quality Markdown with a stable executive summary, structured diagnosis, source/privacy classification, ordered configuration, failing cases, evidence diagnosis, failure labels, comparison changes, prioritized recommendations, and sharing note. The renderer escapes user-controlled Markdown, blocks raw HTML, re-applies the 280-character snippet cap, omits content-bearing fields in metadata-only mode, and rejects full-local exports. Exact output is protected by checked-in trace, eval, CI, metadata-only, and snippets-allowed fixtures.

Privacy And Security Model

Default behavior is local and privacy-first.

- Uploaded binaries are not stored.
- Extracted chunk text is stored in Postgres.
- Embeddings are stored in Postgres.
- No hosted LLM or hosted embedding API is called in the current retrieval path.
- `/api/v1/config` exposes safe config only; database URLs and deployment secrets stay server-side.

Local auth now implements the first hosted boundary: signup/login/logout/current-user, organizations, workspaces, workspace memberships, opaque HttpOnly session cookies, and workspace-scoped API keys. API key secrets use a `clab_...` prefix, are shown once, and are stored only as SHA-256 hashes. Workbench APIs require a session; CI eval routes require a key with the `ci_eval_runs` scope.

`docs/privacy-review-checklist.md` defines the mandatory review gate for data movement, external providers, hosted features, auth, retention, sharing, exports, and telemetry. `docs/logging-redaction.md` defines an allowlist for safe structured metadata and prohibits raw corpus/query content, vectors, credentials, headers, cookies, and secret hashes. Queries are sensitive by default, and future hosted sync must show and redact its payload before crossing the local boundary.

The current logging audit found one API startup event containing bind address, environment, and storage backend kind. Request bodies and sensitive RAG/auth data are not logged. Adding request tracing requires route-template logging and explicit sensitive-header handling.

Hosted mode will still need tenant isolation hardening, invitations, SSO/SAML, SCIM, audit events, upload scanning, and configurable data retention.

Guided Demo Workflow

The guided demo is a sellable-workflow proof that exercises the complete local product path without resetting a workspace. `GET /api/v1/demo` returns persisted progress and a typed suggested-query catalog; `POST /api/v1/demo/load` idempotently creates or repairs three synthetic documents.

`apps/api/src/ingestion.rs` is shared by browser uploads and compiled fixtures, so extraction, document intelligence, structured chunking, checksums, duplicate detection, and quality flags cannot drift between demo and production ingestion. The demo uses 128 target tokens, 16 overlap tokens, and the version marker `corpuslab-guided-demo-v1`.

`DemoRepository` extends both storage adapters with workspace-scoped deterministic project/source/document/chunk upserts and source-specific latest retrieval/trace lookup. IDs use SHA-256-derived UUIDs. Progress is reconstructed from normal persisted data: expected documents, chunks, embeddings, a source-matching retrieval run, its trace, and a trace-sourced audit report.

The web Home route is implemented under `features/workbench/home` and exposes one next action at a time. Retrieval receives only `demo_query=<stable-id>`, resolves question text through the authenticated demo contract, and automatically selects the demo source. Report Markdown can be copied or downloaded unless privacy is `full_local_only`.

See `docs/guided-demo.md` for the exact walkthrough, versioning policy, privacy analysis, and operational troubleshooting.

Configuration Model

`crates/core/src/config.rs` defines:

- `ProductConfig`
- `IngestionConfig`
- `ChunkingConfig`
- `RetrievalConfig`
- `RetrievalWeights`
- `EmbeddingConfig`
- `UiConfig`
- `AuthConfig`

`apps/api/src/config.rs` loads environment values, validates numeric fields, and exposes safe config through `GET /api/v1/config`.

All major defaults should be changed through `.env.example` values rather than hidden route constants.

Testing Strategy

Rust:

```
cargo fmt --all --check
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace
cargo build --workspace
```

Web:

```
cd apps/web
npm run typecheck
npm run lint
npm test
npm run build
npm run format:check
```

Full local check:

```
just check
```

Focused and release-equivalent gates:

```
just rust-check
just web-check
just ci-check
```

Workbench UI quality is enforced through a typed Playwright route matrix at 1440, 1280, 1024, 768, and 390 pixel widths. It verifies semantic headings, primary or empty-state actions, active navigation, keyboard behavior, reduced motion, route isolation, overlap, page overflow, and child-to-panel containment. Synthetic stress fixtures exercise populated retrieval, document, trace, experiment, report, and API-key surfaces with long technical content.

Review screenshots are generated locally with `npm run screenshots:workbench` and remain in ignored Playwright output. The review checklist and assertion policy are documented in `docs/workbench-ui-quality.md`.

`just full-check` remains a backward-compatible alias for `just ci-check`. Documentation ownership, ADR triggers, and changelog expectations are defined in `docs/doc-maintenance.md`. Generated output is excluded unless intentionally versioned; the handbook PDF and curated product assets are explicit exceptions.

Focused RAG invariant validation:

```
cargo test -p rag-debugger-rag
cargo test -p rag-debugger-rag --test public_fixtures
```

Handbook PDF:

```
just docs-pdf
```

Roadmap To Hosted Team Product

The architecture should grow toward:

- Hardened organizations and workspaces.
- Users, roles, and invitations.
- API keys, scoped service accounts, and CI release gates.
- Local collector for private networks.
- Hosted control plane for dashboards, reports, and team collaboration.
- Worker queue for ingestion, embedding, OCR, reranking, and reports.
- Audit events for uploads, queries, reports, and config changes.
- Shareable reports with redaction controls.

Roadmap To GPU And HPC Workers

The future HPC path should stay cleanly separated from the API route layer:

- Add a worker crate for embedding and reranking jobs.
- Introduce job queues and worker leases.
- Add ONNX/Candle model loading for local embeddings.

- Add CUDA and Metal provider implementations behind the embedding provider trait.
- Add batch embedding throughput metrics.
- Add corpus-scale indexing benchmarks.
- Add vector index backends such as pgvector, Tantivy, or a dedicated ANN service.

The goal is not just faster retrieval. The goal is explainable retrieval quality at corpus scale.